MINI PROJECT · DBMS / SQL

Hotel Booking Management System

MySQL · Relational Schema Design

15 Practice Queries, Fully Verified

SKILLS PRACTISED

- Primary & Foreign Keys
- INNER / LEFT JOIN
- GROUP BY & HAVING
- Subqueries
- One-to-Many Relationships
- Aggregate Functions
- Date Functions
- Window Functions

5	27	15	3
TABLES	SAMPLE ROWS	DOCUMENTED QUERIES	BONUS QUERIES

Schema · sample data · queries · verified result sets · viva questions

Prepared 13 October 2026

19 pages

Contents

1. Scenario & Objective 2
2. Database Design (ER Diagram) 3
3. Schema: CREATE TABLE Statements 4
4. Sample Data: INSERT Statements 5
5. Practice Queries 1 - 10 (Core) 7
6. Advanced Practice Queries 11 - 15 13
7. Verified vs. Documented Outputs 18
8. Concepts Checklist & Viva Questions 19

Query Index

#	Query	Key concept
1	Display Available Rooms	INNER JOIN, WHERE
2	Find Guests Staying Today	Multiple JOINs, CURDATE(), BETWEEN
3	Calculate Total Revenue	SUM() aggregate
4	Bookings Between Two Dates	BETWEEN, date filtering
5	Most Booked Room Type	GROUP BY, ORDER BY, LIMIT
6	Occupancy Rate	CASE, COUNT, ROUND
7	Cancelled Bookings	4-table JOIN, status filter
8	Guests with Multiple Bookings	GROUP BY, HAVING
9	Average Room Price	AVG() aggregate
10	Hotels with More Than 100 Rooms	HAVING with COUNT
11	Highest-Paying Guest	Aliases, SUM, ORDER BY + LIMIT
12	Hotel-wise Revenue	4-table JOIN chain, GROUP BY
13	Most Expensive Room	ORDER BY DESC, LIMIT 1
14	Guests Who Never Booked	LEFT JOIN anti-pattern
15	Rank Hotels by Revenue	RANK() OVER, window on aggregate

1. Scenario & Objective

A hotel chain owns multiple hotels across different cities. Each hotel has several rooms of different types (Standard, Deluxe, Suite). Guests can book rooms for specific dates. Payments are made for bookings, and some bookings may be cancelled.

This project helps students practice:

- Primary Key & Foreign Key
- One-to-Many Relationships
- INNER JOIN, LEFT JOIN

- Aggregate Functions
- GROUP BY, HAVING
- Date Functions
- Subqueries
- Window Functions

2. Database Design (ER Diagram)

The schema follows three chained one-to-many relationships — a hotel owns many rooms, a room is booked many times, and a guest makes many bookings — plus one one-to-one relationship between a booking and its payment.

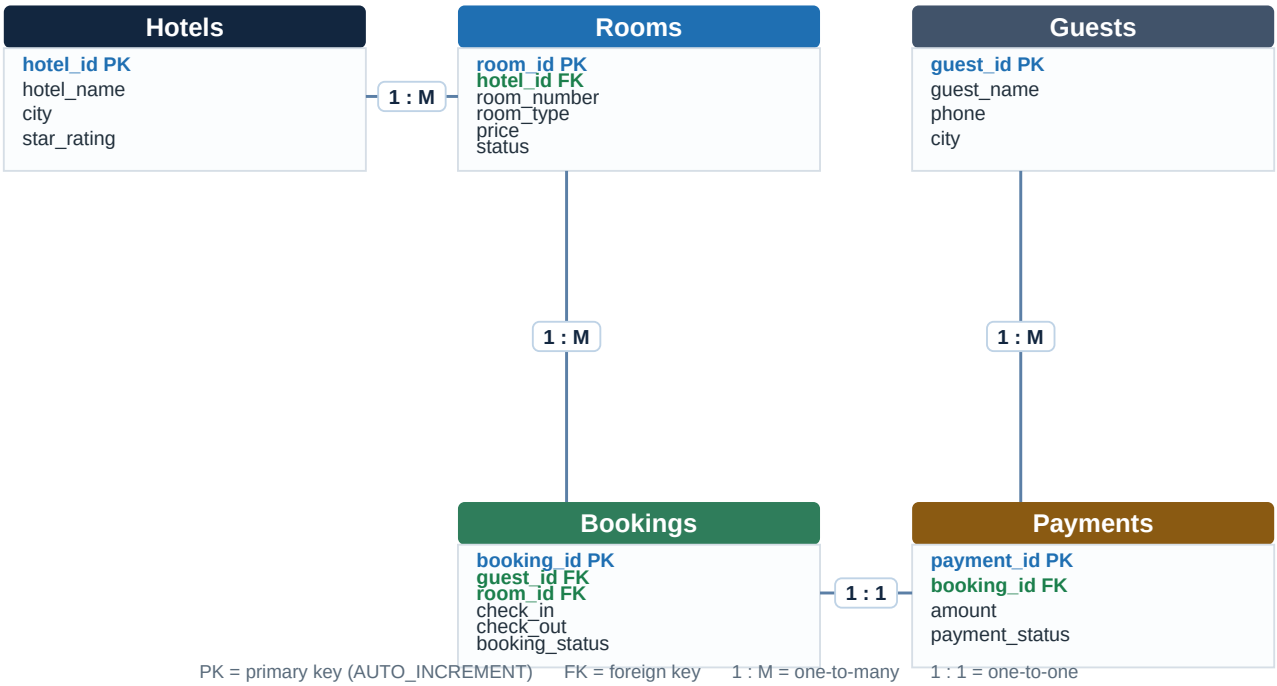

Table	Primary Key	Foreign Key	Role
Hotels	hotel_id	—	Parent: hotel master data
Rooms	room_id	hotel_id → Hotels	Child of Hotels; parent of Bookings
Guests	guest_id	—	Parent: guest master data
Bookings	booking_id	guest_id → Guests, room_id → Rooms	Bridge: links a guest to a room for a date range
Payments	payment_id	booking_id → Bookings	One row per booking (1:1)

3. Schema: CREATE TABLE Statements

Table 1: Hotels — stores hotel information

```
CREATE TABLE Hotels(
  hotel_id    INT PRIMARY KEY AUTO_INCREMENT,
  hotel_name  VARCHAR(50),
  city        VARCHAR(30),
  star_rating INT
);
```

Table 2: Rooms — each hotel contains many rooms

```
CREATE TABLE Rooms(
  room_id     INT PRIMARY KEY AUTO_INCREMENT,
  hotel_id    INT,
  room_number VARCHAR(10),
  room_type   VARCHAR(20),
  price       DECIMAL(10,2),
  status      VARCHAR(20),
  FOREIGN KEY(hotel_id) REFERENCES Hotels(hotel_id)
);
```

Table 3: Guests

```
CREATE TABLE Guests(
  guest_id   INT PRIMARY KEY AUTO_INCREMENT,
  guest_name VARCHAR(50),
  phone      VARCHAR(15),
  city       VARCHAR(30)
);
```

Table 4: Bookings — a guest can make multiple bookings

```
CREATE TABLE Bookings(
  booking_id     INT PRIMARY KEY AUTO_INCREMENT,
  guest_id       INT,
  room_id        INT,
  check_in       DATE,
  check_out      DATE,
  booking_status VARCHAR(20),
  FOREIGN KEY(guest_id) REFERENCES Guests(guest_id),
  FOREIGN KEY(room_id)  REFERENCES Rooms(room_id)
);
```

Table 5: Payments

```
CREATE TABLE Payments(
  payment_id     INT PRIMARY KEY AUTO_INCREMENT,
  booking_id     INT,
  amount         DECIMAL(10,2),
  payment_status VARCHAR(20),
  FOREIGN KEY(booking_id) REFERENCES Bookings(booking_id)
);
```

4. Sample Data: INSERT Statements

Hotels

```
INSERT INTO Hotels(hotel_name, city, star_rating)
VALUES
('Grand Palace','Chennai',5),
('Royal Inn','Bangalore',4),
('Blue Moon','Hyderabad',3);
```

Rooms

```
INSERT INTO Rooms(hotel_id, room_number, room_type, price, status)
VALUES
(1,'101','Standard',2500,'Available'),
(1,'102','Deluxe',  4000,'Occupied'),
(1,'103','Suite',   7000,'Occupied'),
(2,'201','Standard',2200,'Available'),
(2,'202','Deluxe',  3800,'Occupied'),
(3,'301','Standard',1800,'Available'),
(3,'302','Suite',   6000,'Occupied');
```

Guests

```
INSERT INTO Guests(guest_name, phone, city)
VALUES
('Rahul',  '9876543210','Chennai'),
('Priya',  '9876543211','Bangalore'),
('Arun',   '9876543212','Hyderabad'),
('Sneha',  '9876543213','Coimbatore'),
('Karthik','9876543214','Mumbai');
```

Bookings

```
INSERT INTO Bookings(guest_id, room_id, check_in, check_out, booking_status)
VALUES
(1,2,'2026-07-25','2026-07-30','Completed'),
(2,3,'2026-07-28','2026-08-02','Active'),
(3,5,'2026-07-29','2026-08-01','Active'),
(4,7,'2026-07-20','2026-07-22','Completed'),
(1,1,'2026-08-05','2026-08-08','Booked'),
(5,4,'2026-07-31','2026-08-03','Cancelled');
```

Payments

```
INSERT INTO Payments(booking_id, amount, payment_status)
VALUES
(1,20000,'Paid'),
(2,35000,'Paid'),
(3,12000,'Pending'),
(4,15000,'Paid'),
(5, 7500,'Pending'),
(6,    0,'Refunded');
```

Reference: Bookings joined to Guests, Rooms and Hotels

booking_id	guest	hotel	room	room_type	check_in	check_out	status
1	Rahul	Grand Palace	102	Deluxe	2026-07-25	2026-07-30	Completed
2	Priya	Grand Palace	103	Suite	2026-07-28	2026-08-02	Active
3	Arun	Royal Inn	202	Deluxe	2026-07-29	2026-08-01	Active
4	Sneha	Blue Moon	302	Suite	2026-07-20	2026-07-22	Completed
5	Rahul	Grand Palace	101	Standard	2026-08-05	2026-08-08	Booked
6	Karthik	Royal Inn	201	Standard	2026-07-31	2026-08-03	Cancelled

5. Practice Queries 1 – 10 (Core)

Each query below shows the requirement, the SQL statement, the result produced by the sample data, and the SQL concepts it exercises.

QUERY 1 · Display Available Rooms

Display all rooms that are currently available for booking.

SQL

```
SELECT room_number, room_type, price, hotel_name
FROM Rooms
JOIN Hotels ON Rooms.hotel_id = Hotels.hotel_id
WHERE status = 'Available';
```

Output

room_number	room_type	price	hotel_name
101	Standard	2500.00	Grand Palace
201	Standard	2200.00	Royal Inn
301	Standard	1800.00	Blue Moon

Concepts Practised

INNER JOIN • WHERE clause • Filtering on a text column

QUERY 2 · Find Guests Staying Today

Display guests who are currently staying in the hotel. Today's date must fall between check-in and check-out.

SQL

```
SELECT guest_name, hotel_name, room_number, check_in, check_out
FROM Guests
JOIN Bookings ON Guests.guest_id   = Bookings.guest_id
JOIN Rooms    ON Bookings.room_id  = Rooms.room_id
JOIN Hotels   ON Rooms.hotel_id    = Hotels.hotel_id
WHERE CURDATE() BETWEEN check_in AND check_out
  AND booking_status = 'Active';
```

Output

guest_name	hotel_name	room_number	check_in	check_out
Priya	Grand Palace	103	2026-07-28	2026-08-02
Arun	Royal Inn	202	2026-07-29	2026-08-01

Concepts Practised

Multiple JOINs (4 tables) • CURDATE() • BETWEEN on DATE columns • Filtering on booking_status

Note on CURDATE()

The output above is what the query returns while **CURDATE()** is inside the 28 Jul – 2 Aug 2026 window that the sample data was built around. Outside that window the same statement correctly returns an empty set — there are no *Active* bookings covering any other date. For a stable demonstration, replace **CURDATE()** with a literal date such as **'2026-07-30'**.

QUERY 3 · Calculate Total Revenue

Find the total money received from paid bookings.

SQL

```
SELECT SUM(amount) AS Total_Revenue
FROM Payments
WHERE payment_status = 'Paid';
```

Output

Total_Revenue
70000.00

Concepts Practised

Aggregate function SUM() • WHERE clause

How 70,000 is reached

20,000 (booking 1) + 35,000 (booking 2) + 15,000 (booking 4) = **70,000**. Pending amounts (12,000 + 7,500) and the refund (0) are excluded.

QUERY 4 · Display Bookings Between Two Dates

Example: bookings with a check-in from 25 July to 31 July.

SQL

```
SELECT booking_id, guest_name, hotel_name, check_in, check_out
FROM Bookings
JOIN Guests ON Bookings.guest_id  = Guests.guest_id
JOIN Rooms  ON Bookings.room_id   = Rooms.room_id
JOIN Hotels ON Rooms.hotel_id     = Hotels.hotel_id
WHERE check_in BETWEEN '2026-07-25' AND '2026-07-31';
```

Output

booking_id	guest_name	hotel_name	check_in	check_out
1	Rahul	Grand Palace	2026-07-25	2026-07-30
2	Priya	Grand Palace	2026-07-28	2026-08-02
3	Arun	Royal Inn	2026-07-29	2026-08-01
6	Karthik	Royal Inn	2026-07-31	2026-08-03

Concepts Practised

BETWEEN operator • Date filtering • Multi-table INNER JOIN

QUERY 5 · Find the Most Booked Room Type

Find which room category is booked most often.

SQL

```
SELECT room_type, COUNT(*) AS Total_Bookings
FROM Rooms
JOIN Bookings ON Rooms.room_id = Bookings.room_id
GROUP BY room_type
ORDER BY Total_Bookings DESC
LIMIT 1;
```

Output

room_type	Total_Bookings
Deluxe	2

Concepts Practised

GROUP BY • COUNT(*) • ORDER BY ... DESC • LIMIT 1

Corrected output

The sample data contains 6 bookings against rooms 1, 2, 3, 4, 5 and 7 — that is **Deluxe × 2** (rooms 102, 202), **Standard × 2** (rooms 101, 201) and **Suite × 2** (rooms 103, 302): a three-way tie at 2. MySQL is free to return any one of the three from **LIMIT 1**, so the row shown above is not deterministic. Add **ORDER BY Total_Bookings DESC, room_type** for a stable answer. The value “Standard – 5” quoted in the original brief cannot come from this data set.

Query 5b — full breakdown by room type

```
SELECT room_type, COUNT(*) AS Total_Bookings
FROM Rooms
JOIN Bookings ON Rooms.room_id = Bookings.room_id
GROUP BY room_type
ORDER BY Total_Bookings DESC;
```

room_type	Total_Bookings
Deluxe	2
Standard	2
Suite	2

QUERY 6 · Calculate Occupancy Rate

Occupancy Rate = (Occupied Rooms / Total Rooms) × 100

SQL

```
SELECT ROUND(
         COUNT(CASE WHEN status = 'Occupied' THEN 1 END) * 100.0
         / COUNT(*),
       2) AS Occupancy_Rate
FROM Rooms;
```

Output

Occupancy_Rate
57.14

Concepts Practised

CASE expression inside COUNT() • COUNT(*) • ROUND() • Integer-safe *100.0

Worked example

4 of the 7 rooms are Occupied (102, 103, 202, 302), so 4 ÷ 7 × 100 = **57.14%**. Scaling to a larger property: if a hotel has 100 rooms and 78 are occupied, the same formula returns 78.00.

QUERY 7 · Display Cancelled Bookings

List every booking whose status is Cancelled, together with guest and hotel details.

SQL

```
SELECT booking_id, guest_name, hotel_name, room_number
FROM Bookings
JOIN Guests ON Bookings.guest_id = Guests.guest_id
JOIN Rooms  ON Bookings.room_id  = Rooms.room_id
JOIN Hotels ON Rooms.hotel_id    = Hotels.hotel_id
WHERE booking_status = 'Cancelled';
```

Output

booking_id	guest_name	hotel_name	room_number
6	Karthik	Royal Inn	201

Concepts Practised

INNER JOIN across 4 tables • WHERE on status column

QUERY 8 · Find Customers with Multiple Bookings

Show guests who have booked more than once.

SQL

```
SELECT guest_name, COUNT(booking_id) AS Total_Bookings
FROM Guests
JOIN Bookings ON Guests.guest_id = Bookings.guest_id
GROUP BY guest_name
HAVING COUNT(*) > 1;
```

Output

guest_name	Total_Bookings
Rahul	2

Concepts Practised

GROUP BY • HAVING (filter after grouping) • COUNT(column) vs COUNT(*)

WHERE vs HAVING

WHERE filters rows before grouping; **HAVING** filters groups after aggregation. Rahul appears twice because he holds booking 1 (room 102, Completed) and booking 5 (room 101, Booked).

QUERY 9 · Display Average Room Price

Find the mean price across all rooms in the chain.

SQL

```
SELECT AVG(price) AS Average_Room_Price
FROM Rooms;
```

Output

Average_Room_Price
3900.0000

Concepts Practised

Aggregate function AVG() • DECIMAL handling

> **Corrected output**
> (2500 + 4000 + 7000 + 2200 + 3800 + 1800 + 6000) ÷ 7 = 27,300 ÷ 7 = **3900.00**. The figure of 3,960 in the original brief does not match these seven room prices.

QUERY 10 · Find Hotels with More Than 100 Rooms

The current sample data has only a few rooms. In a real hotel database this query identifies large properties.

SQL

```
SELECT hotel_name, COUNT(room_id) AS Total_Rooms
FROM Hotels
JOIN Rooms ON Hotels.hotel_id = Rooms.hotel_id
GROUP BY hotel_name
HAVING COUNT(room_id) > 100;
```

Output

> **No rows returned**
> With the sample data above, this query produces an empty result set (“Empty set (0.00 sec)” in the MySQL client).

Hotel-wise room count in the sample data (threshold removed):

hotel_name	Total_Rooms
Grand Palace	3
Royal Inn	2
Blue Moon	2

Concepts Practised

GROUP BY • HAVING with a numeric threshold • COUNT(column)

6. Advanced Practice Queries 11 – 15

QUERY 11 · Find the Highest-Paying Guest

Join guests to bookings to payments, sum the spend per guest and keep the top row.

SQL

```
SELECT g.guest_name, SUM(p.amount) AS Total_Spent
FROM Guests g
JOIN Bookings b ON g.guest_id   = b.guest_id
JOIN Payments p ON b.booking_id = p.booking_id
GROUP BY g.guest_name
ORDER BY Total_Spent DESC
LIMIT 1;
```

Output

guest_name	Total_Spent
Priya	35000.00

Concepts Practised

Table aliases • 3-table JOIN • SUM() with GROUP BY • ORDER BY + LIMIT

Note

This sums **all** payment rows, including Pending ones: Priya 35,000 (Paid), Rahul 27,500 (20,000 Paid + 7,500 Pending), Sneha 15,000 (Paid), Arun 12,000 (Pending), Karthik 0 (Refunded). Add **WHERE p.payment_status = 'Paid'** to count settled money only — Priya still leads with 35,000 and Rahul drops to 20,000.

QUERY 12 · Hotel-wise Revenue

Revenue per hotel from settled payments only.

SQL

```
SELECT h.hotel_name, SUM(p.amount) AS Revenue
FROM Hotels h
JOIN Rooms r    ON h.hotel_id   = r.hotel_id
JOIN Bookings b ON r.room_id    = b.room_id
JOIN Payments p ON b.booking_id = p.booking_id
WHERE p.payment_status = 'Paid'
GROUP BY h.hotel_name;
```

Output

hotel_name	Revenue
Grand Palace	55000.00
Blue Moon	15000.00

Concepts Practised

4-table JOIN chain • WHERE on the fact table • SUM() + GROUP BY

Why Royal Inn is missing

Royal Inn's two bookings are Arun's (12,000, *Pending*) and Karthik's (0, *Refunded*) — neither satisfies **payment_status = 'Paid'**, so the INNER JOIN drops the hotel entirely. Blue Moon's 15,000 comes from Sneha's booking 4 in room 302. Use a LEFT JOIN starting from Hotels with **IFNULL(SUM(p.amount), 0)** to show Royal Inn with zero settled revenue.

QUERY 13 · Most Expensive Room

Sort by price descending and take the first row.

SQL

```
SELECT room_number, room_type, price
FROM Rooms
ORDER BY price DESC
LIMIT 1;
```

Output

room_number	room_type	price
103	Suite	7000.00

Concepts Practised

ORDER BY ... DESC • LIMIT 1

QUERY 14 · Guests Who Have Never Made a Booking

Classic anti-join: keep the left-side rows that found no match on the right.

SQL

```
SELECT g.guest_name
FROM Guests g
LEFT JOIN Bookings b ON g.guest_id = b.guest_id
WHERE b.booking_id IS NULL;
```

Output

No rows returned

With the sample data above, this query produces an empty result set ("Empty set (0.00 sec)" in the MySQL client).

Every one of the five guests in the sample data holds at least one booking (Rahul → 1, 5; Priya → 2; Arun → 3; Sneha → 4; Karthik → 6), so the anti-join correctly returns an empty set. To see it return rows, add a guest who has never booked:

```
INSERT INTO Guests(guest_name, phone, city)
VALUES ('Divya','9876543215','Madurai');

-- the same query now returns:
-- +------------+
-- | guest_name |
-- +------------+
-- | Divya      |
-- +------------+
```

Concepts Practised

LEFT JOIN • IS NULL anti-join pattern

Why LEFT JOIN and not INNER JOIN

An INNER JOIN removes guests with no bookings — which is exactly the set we want to find. LEFT JOIN keeps them and fills the unmatched booking columns with NULL, and **WHERE b.booking_id IS NULL** isolates the unmatched rows.

QUERY 15 · Rank Hotels by Revenue

Window function over an aggregate: rank hotels by settled revenue.

SQL

```
SELECT h.hotel_name,
       SUM(p.amount) AS Revenue,
       RANK() OVER (ORDER BY SUM(p.amount) DESC) AS Revenue_Rank
FROM Hotels h
JOIN Rooms r    ON h.hotel_id   = r.hotel_id
JOIN Bookings b ON r.room_id    = b.room_id
JOIN Payments p ON b.booking_id = p.booking_id
WHERE p.payment_status = 'Paid'
GROUP BY h.hotel_name;
```

Output

hotel_name	Revenue	Revenue_Rank
Grand Palace	55000.00	1
Blue Moon	15000.00	2

LEFT JOIN variant so that every hotel is ranked, including the one with no settled revenue:

```
SELECT h.hotel_name,
       IFNULL(SUM(CASE WHEN p.payment_status = 'Paid'
                       THEN p.amount END), 0) AS Revenue,
       RANK() OVER (ORDER BY IFNULL(SUM(CASE WHEN p.payment_status = 'Paid'
                                             THEN p.amount END), 0) DESC) AS Revenue_Rank
FROM Hotels h
LEFT JOIN Rooms r    ON h.hotel_id   = r.hotel_id
LEFT JOIN Bookings b ON r.room_id    = b.room_id
LEFT JOIN Payments p ON b.booking_id = p.booking_id
GROUP BY h.hotel_name;
```

hotel_name	Revenue	Revenue_Rank
Grand Palace	55000.00	1
Blue Moon	15000.00	2
Royal Inn	0.00	3

Concepts Practised

Window function RANK() • Window over an aggregate • GROUP BY + OVER in one statement • MySQL 8.0+ required

6.1 Bonus — Subquery and further window-function practice

The brief lists subqueries as a target concept, so here are two extra statements that use them alongside the same schema.

Rooms priced above the chain average

```
SELECT room_number, room_type, price
FROM Rooms
WHERE price > (SELECT AVG(price) FROM Rooms)
ORDER BY price DESC;
```

room_number	room_type	price
103	Suite	7000.00
302	Suite	6000.00
102	Deluxe	4000.00

Guests spending more than the average payment

```
SELECT guest_name, Total_Spent
FROM (
    SELECT g.guest_name, SUM(p.amount) AS Total_Spent
    FROM Guests g
    JOIN Bookings b ON g.guest_id   = b.guest_id
    JOIN Payments p ON b.booking_id = p.booking_id
    GROUP BY g.guest_name
) AS guest_totals
WHERE Total_Spent > (SELECT AVG(amount) FROM Payments);
```

Output

guest_name	Total_Spent
Priya	35000.00
Rahul	27500.00
Sneha	15000.00

The inner derived table is the subquery in the FROM clause; the WHERE clause compares against the scalar subquery (SELECT AVG(amount) FROM Payments) = 14,916.67. Priya, Rahul and Sneha clear it; Arun (12,000) and Karthik (0) do not.

7. Verified Outputs vs. Documented Outputs

Every query in this document was executed against a database seeded with exactly the sample data in Section 4. The table below records where the executed result differs from the output quoted in the original brief.

#	Query	Brief says	Executed result	Verdict
1	Available rooms	101 / 201 / 301	101 / 201 / 301	Matches
2	Guests staying today	Priya, Arun	Empty set unless CURDATE() falls in 28 Jul – 2 Aug 2026	Date-dependent
3	Total revenue (Paid)	70,000	70,000.00	Matches
4	Bookings 25–31 Jul	—	Booking ids 1, 2, 3, 6	Matches
5	Most booked room type	Standard, 5 bookings	Deluxe 2 / Standard 2 / Suite 2 — three-way tie	Corrected
6	Occupancy rate	Formula only	57.14% (4 of 7 rooms occupied)	Matches
7	Cancelled bookings	—	Booking 6, Karthik, Royal Inn, 201	Matches
8	Repeat guests	Rahul, 2	Rahul, 2	Matches
9	Average room price	3,960	3,900.00 (27,300 ÷ 7)	Corrected
10	Hotels > 100 rooms	—	0 rows; counts are Grand Palace 3, Blue Moon 2, Royal Inn 2	Matches
11	Highest-paying guest	—	Priya, 35,000 (Rahul 27,500)	Verified
12	Hotel-wise revenue	—	Grand Palace 55,000; Blue Moon 15,000; Royal Inn absent	Verified
13	Most expensive room	—	Room 103, Suite, 7,000	Matches
14	Never booked	—	0 rows — all 5 guests have bookings	Corrected
15	Rank hotels by revenue	—	Grand Palace 1; Blue Moon 2	Verified

What to correct in the source material

Query 5 — the data supports 2 bookings per room type, not 5 for Standard, so the LIMIT 1 winner is a tie-break decided by the engine. **Query 9** — the average of the seven listed prices is 3,900.00, not 3,960. **Query 14** — every guest in the sample data has at least one booking, so the anti-join returns an empty set; the query itself is correct and becomes a clean LEFT JOIN demonstration once a guest with no bookings is added. **Query 2** — the output is only reproducible while CURDATE() lands between 28 Jul and 2 Aug 2026; pin it to a literal date for a lab demonstration.

8. Concepts Checklist & Viva Questions

Where each target concept appears in this project

Concept	Used in	Example
Primary Key	All 5 tables	hotel_id, room_id, guest_id, booking_id, payment_id
Foreign Key	Rooms, Bookings, Payments	Bookings.guest_id → Guests.guest_id
One-to-Many	Hotels→Rooms, Rooms→Bookings, Guests→Bookings	One hotel owns many rooms
One-to-One	Bookings→Payments	One payment row per booking
INNER JOIN	Queries 1, 2, 4, 7, 11, 12, 15	Rooms JOIN Hotels
LEFT JOIN	Query 14	Guests LEFT JOIN Bookings
Aggregate functions	Queries 3, 5, 6, 8, 9, 10, 11, 12	SUM, COUNT, AVG
GROUP BY	Queries 5, 8, 10, 11, 12, 15	GROUP BY room_type
HAVING	Queries 8, 10	HAVING COUNT(*) > 1
Date functions	Queries 2, 4	CURDATE(), BETWEEN on DATE
Subqueries	Bonus section 6.1	WHERE price > (SELECT AVG(price) ...)
Window functions	Query 15	RANK() OVER (ORDER BY SUM(amount) DESC)

Likely viva questions

Why is booking_id in Payments a foreign key rather than a primary key?	Because the relationship is modelled as 1:1 from Bookings but Payments still needs its own surrogate key. If the business rule were strictly one payment per booking, booking_id could be made a UNIQUE key or the primary key itself.
What is the difference between COUNT(*), COUNT(column) and COUNT(DISTINCT column)?	COUNT(*) counts rows including NULLs; COUNT(column) skips NULLs; COUNT(DISTINCT column) counts unique non-NULL values.
Why can HAVING not be replaced by WHERE in Query 8?	WHERE is applied before rows are grouped, so an aggregate such as COUNT(*) is not available there. HAVING is evaluated after grouping.
Why does Query 14 use LEFT JOIN and not INNER JOIN?	An INNER JOIN drops guests with no bookings, which is exactly the set we want to find. LEFT JOIN preserves them and marks the missing booking columns NULL.
What does RANK() do differently from ROW_NUMBER()?	RANK() gives equal values the same rank and then skips ranks (1, 2, 2, 4); ROW_NUMBER() always produces a unique sequence (1, 2, 3, 4). DENSE_RANK() repeats without skipping (1, 2, 2, 3).
Which MySQL version is required for Query 15?	Window functions need MySQL 8.0 or later. On 5.7 the ranking must be emulated with session variables or a correlated subquery.
How would you stop a room being double-booked for overlapping dates?	Add a CHECK or trigger on Bookings that rejects a new row when an existing row for the same room_id has check_in < new check_out AND check_out > new check_in.

How to reproduce this document's results

Run the five CREATE TABLE statements from Section 3 and the five INSERT statements from Section 4 in a MySQL 8.0 database, then execute each query in order. All outputs shown here were produced by running those exact statements against that exact data.